

IoT-Based Smart Shelf Microclimate Monitoring and Spoilage Risk Analytics System for MSME Retailers

MS.Yashaswini

Computer Science and Technology
Assistant Professor
Dayananda Sagar University
Bangalore, India
yashaswini-ct@dsu.edu.in

G.S Prajwal

Computer Science and Technology
Dayananda Sagar University
Bangalore, India
eng23ct0004@dsu.edu.in

P.Jeevan Kumar Reddy

Computer Science and Technology
Dayananda Sagar University
Bangalore, India
eng23ct0036@dsu.edu.in

E.Yashas kumar

Computer Science and Technology
Dayananda Sagar University
Bangalore, India
eng23ct0002@dsu.edu.in

Abstract—The spoilage of fresh raw produce continues to pose a significant and ongoing economic burden for micro, small, and medium sized retail firms (MSMEs), including local (Kirana) grocery stores and vegetable merchants, as the majority of these businesses have no way to continuously or accurately monitor the temperature, humidity, and volatile gas characteristics that increase the speed of spoilage (i.e., deterioration). Large industrial (i.e., manufacturer) classification-based solutions such as passive RFID (radio frequency identification) devices connected to multi-sensor networks that use ultra-high frequency (UHF) RFID reader devices and battery-powered tags, with a cloud-hosted machine learning inference engine, are out of reach of MSME retailers' budgets, and the presence of moisture-dense produce has a negative impact on the effective range of passive RFID.

This work proposes a low-cost Internet of Things (IoT)-based Smart Shelf Microclimate Monitoring and Spoilage Risk Analytics System developed specifically for MSME retail applications. The solution combines a DHT22 digital humidity/temperature sensor with an MQ-135 volatile organic compound (VOC) gas sensor connected to a low-cost ESP32 microcontroller. The solution's lightweight, rules-based spoilage risk index algorithm runs entirely on-device, allowing for the autonomous triggering of an external relay-controlled exhaust fan to address unfavorable microclimatic conditions without relying on continuous cloud connectivity. Telemetry data captures are stored on a MongoDB Atlas database on a FastAPI backend and displayed operationally on a dashboard to perform real-time monitoring, retrospectively review data, and to forecast potential risks within the next hour using moving average smoothing algorithms. All hardware components have been developed to stay below 2,000 (INR) in material costs while the anticipated monthly recurring cost for electricity is 9.55/month (INR)—several orders of magnitude lower than industrial cold chain or RFID-based solutions. The proposed solution will eliminate radio frequency item level sensing as well as making on premises decision-making dependent solely on the cloud providing a solution to the previously identified void between passive environmental logging devices and

offline enabled, automated microclimate calibration of ambient (unphased) retail shelf products.

Index Terms—IoT, edge computing, spoilage risk index, ESP32, MQ-135, smart shelf, MSME retail, microclimate monitoring

I. INTRODUCTION

Small-scale food retail, including kirana stores, open vegetable stalls, and other MSME outlets, forms the backbone of fresh produce distribution across India. However, these outlets often lack the technical means to manage the environmental conditions that lead to spoilage. Field studies of both organized and unorganized retail outlets have identified inadequate storage facilities and improper handling, rather than produce quality alone, as major contributors to shrinkage and spoilage loss at the retail level [1]. Elevated temperatures speed up microbial growth and respiration rates. Low humidity causes wilting and loss of fresh weight. Additionally, volatile organic compounds (VOCs) released during ripening and decay worsen deterioration when air circulation is poor.

Large-scale industrial solutions to this problem do exist, but they do not apply to the MSME context. For example, RFID-powered multi-sensing systems combine passive and semi-passive sensor tags with cloud-hosted predictive models to achieve high-precision, non-destructive quality assessment. One such system achieved a root mean square error of 0.007 in predicting food quality indicators using a NARX neural network [2]. However, these setups rely on specialized UHF reader hardware, battery-powered tags, and continuous data pipelines to cloud inference engines. All of these factors come with capital and maintenance costs that are far beyond what an informal retailer can afford. The same underlying limitation, the vulnerability of RF-based sensing to moisture, has been

confirmed through controlled testing. This testing shows that UHF RFID read range can drop from over 16 m to under 2 m when liquid or moisture-rich contents are present [3]. This makes direct item-level RF sensing unsuitable for producing shelves.

The rise of low-cost, Wi-Fi-enabled microcontrollers like the ESP32 offers a more accessible solution. They allow for environmental sensing and on-device decision-making without needing industrial infrastructure. However, as this paper's analysis highlights in Section II, existing low-cost systems often fall into two unsatisfactory categories. They either act as passive cloud loggers without local action or require complex machine-learning processes (including TinyML) that add training, deployment, and computing burdens that are not proportional to the problems being solved.

In this context, this paper proposes a smart shelf microclimate monitoring and spoilage risk analytics system designed specifically for MSME affordability. The system's main goals are to (i) continuously monitor shelf conditions, (ii) detect unfavorable microclimatic conditions using a combined temperature, humidity, and VOC signal, (iii) autonomously adjust those conditions with a relay-driven exhaust fan without needing network connectivity, (iv) calculate an understandable, tiered spoilage risk score instead of relying on exact shelf-life predictions, and (v) give retailers historical data and short-term forecasts to help with inventory and pricing decisions. The rest of this paper is structured as follows: Section II summarizes the research gaps found in current literature; Section III reviews relevant work in four areas—RFID/multisensor fusion, MQ-series gas sensing, rule-based versus ML risk scoring, and predictive time-series forecasting; Section IV outlines the proposed system architecture; Section V details the implementation; Section VI presents the evaluation results; and Section VII wraps up the paper.

II. RESEARCH GAP SYNTHESIS

Three concrete, literature-evidenced gaps motivate the proposed system design.

A. The Cost and Infrastructure Gap in Pervasive Sensing

Recent industrial RFID-multisensor systems are advancing towards more capable item-level tags. These include rechargeable semi-passive tags that feed cloud LSTM inference [4] and NARX-based quality prediction validated on actual food products [5]. However, each of these methods shares RFID's basic weakness to moisture-related signal loss [3] and relies on either battery upkeep or costly reader infrastructure. Even the closest low-cost alternative, an ESP32/DHT22/MQ-135 onion storage unit aimed at Indian smallholder farmers, is priced at INR 60,000–70,000 [7]. This cost is significantly higher than what an MSME retailer would consider for a single shelf. No available system provides an integrated, sub-INR 2,000 microclimate monitoring platform for active, localized food preservation at the retail level.

B. The Offline Autonomy Gap in Environmental Control

Most environmental monitors that are not expensive just collect data or show information on a screen they do not control things automatically. There is a sensor node that uses ESP32 and wireless technology it can work with sensors but it is made for many different uses, not just for keeping food from spoiling and it does not have special controls for that. A new system that uses sensors to track how ripe bananas are just gives information it does not do anything to fix the problem.

This is a problem: if the internet connection is lost these systems cannot take action quickly. Research on edge computing shows that it is possible to make decisions using rules and machine learning at the edge of the network. For example in irrigation systems that use rules and a special kind of classifier and in environmental sensors that use machine learning to combine gas and VOC data.. No system does this and also controls relays and fans automatically for retail produce shelves. Environmental monitors and sensor nodes like the ESP32-based wireless sensor node are not enough they need to be able to control things, like relays and fans to keep food from spoiling.

C. Fused Volatile and Ambient Analytical Modeling on the Edge, Without an ML Pipeline

The MQ-series gas sensors are really good at showing when fruit is going bad or is ripe. They match up well with how ripe the fruit's things like how hard it is and how much sugar it has in mangoes.. The systems we have now need to either use machine learning on a computer or have a special system that can learn and work on its own on a tiny device. For example there is a system that uses gas sensors and can tell if something is fresh 91.9 percent of the time on a small device called an Arduino Nano 33 BLE Sense.

When we look at all the ways to predict how long something will last and use machine learning on devices we see that using artificial intelligence has some downsides. It needs a lot of data and power. It can take a long time to get the right answers. Even the advanced system that uses deep learning to detect food spoilage still uses a simple rule-based system to make decisions. This shows that we can use a system that combines gas and environment data without machine learning and this is a good choice, for devices that need to work in real-time.

All these things show that the system we are proposing is right where it needs to be. It is a low-cost system that uses an ESP32 device and combines temperature, humidity and gas data to create a Spoilage Risk Index. It can then use this index to control a relay and fix the problem on its own all without needing the cloud, batteries, special sensors or machine learning.

III. LITERATURE SURVEY

A. RFID and Multisensor Fusion IoT Systems for Food Quality/Freshness Sensing

Industrial-grade RFID multisensor research has advanced considerably in tag design and inference sophistication. Cao,

Wu, and Gray developed a semi-passive UHF RFID multisensor tag with a rechargeable lithium battery, extending readable range to 10 m while feeding sensor data to a cloud-hosted LSTM model for real-time food quality Research by Cao in 2026 has led to a prediction that can be directly linked to this concept. research group's foundational RFID-powered multisensing architecture validated on dry-cured ham using a NARX neural network with an RMSE of 0.007 [2], and to a companion study applying AI-assisted RFID tag design to quality assessment more broadly [5]. Independently, Claucherty et al. demonstrated that beverage and liquid content substantially degrades UHF RFID performance, collapsing read range from 16.34 m to under 1.88 m in the presence of water-based solutions [3], establishing a physical limitation intrinsic to RF sensing near moisture-dense produce. More recently, Andre et al. proposed a wireless RF sensor using a carbon-nanotube chemiresistive composite that changes its signal response to volatile amines released during meat spoilage [6], confirming that VOC-driven spoilage detection is a real signal pathway - even if, in this particular case, requires custom nanomaterial fabrication rather than off-the-shelf components. *Synthesis:* Every RFID-based approach surveyed inherits RF's It's pretty basic, but this thing has a major weakness when it comes to moisture, and it needs a battery to work. maintenance, cloud inference, or exotic materials fabrication. None targets shelf-level, rather than item-level, ambient monitoring – the specific gap the proposed system occupies.

B. MQ-Series Gas Sensors as Spoilage or Ripeness Indicators for Fruits/Vegetables

MQ-series metal oxide semiconductor sensors are shown to be good at checking how fresh produce is. They are cheap. Do not hurt the produce. Some people like Trebar, Žalik and Vidrih used these sensors to track the freshness of Golden Delicious apples. They used a few of these sensors and some other methods like PCA, k-means clustering and KNN classification. They were able to tell when the apples were ripe with high accuracy. [12]

Rahman and others made an electronic nose using these sensors for mango farmers in Bangladesh. They found that the sensors could tell how hard the mangoes were and how sugar they had. This showed that these cheap sensors can be used to check how fresh a single type of fruit is ($r = 0.815\text{--}0.847$) [13]

Bhattacharjee and others used these sensors to track how bananas ripen and go bad. They made a display [9]. that showed how good the bananas were.

Synthesis: MQ-135 and similar sensors are good, at checking freshness. They work well with apples, mangoes and bananas.. The systems that use these sensors have some problems. They often need many sensors and special computers to work. They only work with one type of fruit at a time.. They just monitor the fruit and do not do anything to help it stay fresh. This is where our project comes in. We want to

make a system that uses one sensor [12] and can help keep the produce fresh on its own.

C. Rule-Based vs. Machine-Learning Approaches to Spoilage Risk Scoring and Predictive Modeling

Rashvand and his team did a review that compared old math ways to figure out how long something lasts with new machine learning and deep learning ways that use special tools like hyperspectral imaging and sensors. They found out that the new ways that use intelligence need a lot of data and computer power to work well and are hard to use in real life when you do not have a lot of resources. [14]

This is a problem because artificial intelligence methods and machine learning methods need a lot of training data and computation to work well. Rashvand and his team showed this with their review of machine learning methods and artificial intelligence methods.

Some other people like Haque and his team tried to make this work. They used a machine learning model on a special tiny computer called an Arduino Nano 33 BLE Sense with special gas sensors to check if dates are fresh in real time and it worked pretty well with 91.9 percent accuracy [15]. This shows that you can use machine learning on a device like the ESP32 but you have to do a lot of work to get it ready first like training the model and getting it to work on the device.

Another team, Heydari and Mahmoud looked at this problem closely and found out that using machine learning on a small device can make things happen faster from 10 to 500 milliseconds to almost zero milliseconds but it can also make the accuracy a little worse from 95 percent to 85 percent [16] when you use TinyML and on-device inference, with machine learning methods and artificial intelligence methods.

Synthesis: When we look at what people're saying about food spoilage prediction we see that it is like a line with two ends. On one end we have models that do not cost much but they are not very good at predicting when food will go bad. On the end we have models that use machine learning and are very good at predicting when food will go bad but they are also very expensive to use. This is why we chose to use a rule-based model for our food spoilage prediction system, which we call the Spoilage Risk Index. We like this model because it is fast and does not need a lot of resources. At the time we are saving all of the data we get in a database called MongoDB Atlas so that someday we can use it to make even better predictions with machine learning. The Spoilage Risk Index is a choice for us because it is, on the simpler end of the spectrum but we are still getting the data we need to maybe use machine learning later on.

D. Predictive Analytics and Time-Series Forecasting Applied to Food Quality Trends

Recent work, on using computers to predict things shows that simple models are still the best even when we have advanced ways of doing things. R.B.M. And R.S. Did an experiment where they made up 500 sets of sensor readings like temperature and humidity. Then used those readings to

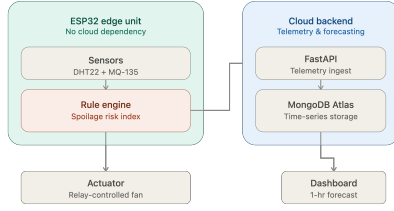


Fig. 1. Two-layer architecture, coupled by one synchronous round trip.

see what would happen. [18] They found out that temperature. How long it takes to transport something are the most important things that can tell us if something will go bad. This is what R.B.M. And R.S. Said in their work, which was published in 2026 and it supports the idea that we should pay attention to temperature first when we are trying to figure out if something will go bad.

In another study Singh and some other people did something complicated. They used a kind of computer program that can make decisions based on rules and they also used a different kind of program that can learn from experience. They tested this system using data and real data from sensors and it worked well. This is what Singh and the other people found out. They published it in 2025 [17]. *Synthesis*: The best systems that use learning to predict when food will go bad still rely on simple rules to make sense of things. These rules are easy to understand [17]. Are based on things, like temperature which is always the main thing that affects how quickly food goes bad. This shows that it is an idea to use a simple average of the risk of food [18] going bad calculated right on the device like we suggest in this paper. This is not a simple solution to save money but it actually makes sense.

IV. PROPOSED SYSTEM ARCHITECTURE

Three constraints from Section II govern the design. *C1 — bounded capital cost*: the node must be purchasable by a single-shelf retailer, ruling out UHF readers and battery-backed tags [4], [5], whose read range degrades near moisture-dense produce [3]. *C2 — no cloud dependency in the safety path*: connectivity cannot be assumed in a kirana store, so any decision protecting stock must be computable from the current sample and a small cached reference set. *C3 — auditable, source-locked biology*: every per-commodity constant must trace to a published postharvest source [19], so commodity biology is versioned reference data, not program text.

A. Layered Decomposition

The **Physical Edge Layer** (Fig. 1) comprises an ESP32 board [8], a DHT22 humidity and temperature sensor, an MQ-135 SnO₂ gas sensor giving the broad VOC and CO₂ trend signal that MQ-series devices have been shown to provide for produce freshness [12], [13], an optocoupler-isolated 5 V relay, and a 5 V brushless exhaust fan venting the enclosure; it samples, transmits raw values, and drives the relay from the command it receives in reply. The **Cloud Analytics Layer**

is an asynchronous FastAPI service over MongoDB Atlas: it resolves biological context, computes the Spoilage Risk Index (SRI), evaluates the actuation hierarchy, maintains alerts, persists the reading, and serves a dashboard showing historical SRI with a one-hour forecast. Coupling is deliberately narrow: one endpoint, `POST /devices/{id}/readings`, accepts a sample and returns the fan command in the same response, with no command channel, broker or polling, so firmware reduces to “sample, post, set relay” and every actuation is reproducible from one stored document.

B. Where the Decision Is Computed

In the implementation evaluated in this paper, the SRI and the fan command are computed server-side, inside that synchronous round trip. The abstract’s characterisation of an entirely on-device rule engine describes the target deployment architecture rather than the build reported here; no ESP32 firmware implementing the SRI exists at the time of writing. The design makes that migration tractable rather than speculative: the decision function depends only on the current sample, six scalar profile fields, one calibration scalar and one boolean of previous state — small enough to cache in ESP32 NVS and evaluate in fixed-point arithmetic, so C2 is satisfiable by porting rather than redesigning (Section VI-E). We state this rather than let the reader infer edge execution from Fig. 1; Section VI-G records it as the principal limitation here.

C. Persistence and Temporal Resolution

Six collections implement the data model: `commodity_profiles`, `devices`, `device_assignments`, `device_calibration`, `readings` and `alerts`. The first, third and fourth are *temporally versioned* rather than mutable — recalibrating a sensor or reassigning a shelf inserts a new row rather than overwriting — so a reading recorded six months ago can still be re-evaluated against the profile and calibration in force when it was taken. Assignment being a separate collection means monitoring history survives reassignment, and `alerts.opened_by_reading_id` records the sample that opened each episode.

Before any arithmetic the service resolves assignment, profile and calibration against the sample’s device timestamp, each lookup followed by a secondary “latest available” query — not redundant, since ESP32 clocks drift and after a power cycle can emit timestamps preceding the seeding of the reference data, leaving a drifted shelf without actuation.

V. IMPLEMENTATION

A. Source-Locked Reference Data

Every per-commodity constant originates in USDA Agriculture Handbook 66 (AH-66) [19], the standard postharvest reference for commercial storage of fruits and vegetables. Values are held in one JSON document carrying page-level citations and projected by a seeding script into `commodity_profiles`; none appears in application code. The separation is enforced by construction: the configuration

TABLE I
COMMODITY REFERENCE CONSTANTS AS SEEDED FROM AH-66 [19]

	Tomato	Onion (cured)	Potato (cured)	Leafy greens
$T_{\min}-T_{\max}$ (°C)	13–21	0–0	7–10	0–0
RH band (%)	90–95	65–75	80–100	95–98
T_{chill} (°C)	13.0	—	2.0	—
T_{ref} (°C)	20	5	20	—
Q_{10} bands ^a	2.38 ^{β} 1.95 ^{δ}	2.33 ^{α} 1.14 ^{β}	1.38 ^{γ} 1.34 ^{β}	5.37 ^{α} 3.97 ^{γ} , 2.09 ^{β}
Respiration @20°C ^b	34.5	8	21.5	229.5
AH-66 pages	581–585	436–439	506–509	353–355

^a α : 0–10, β : 10–20, γ : 5–15, δ : 15–25°C. ^bmg CO₂ kg⁻¹ h⁻¹.
“—” = not tabulated in AH-66.

module holds only engineering tunables, and the SRI routine raises an exception if a resolved profile lacks a required field rather than substituting a default. Seeding is idempotent through a SHA-256 hash of each source block, and the raw block is persisted verbatim beside the parsed fields. Table I gives the four commodities seeded.

Two projection decisions are approximations, not transcriptions: AH-66 gives tomato an RH figure for *ripening* and potato one for *curing* rather than retail display, and leafy-green values are spinach used as a documented proxy; the seeder logs a warning naming each substituted key. Raw MQ-135 ADC counts are stored rather than a derived R_s/R_o ratio, since the ratio is recomputed at evaluation time from the calibration effective at the reading’s timestamp — so recalibration cannot retroactively corrupt history.

B. Spoilage Risk Index

The calibrated gas signal is the ratio of the raw reading to the device’s clean-air baseline B , mapped onto $[0, 1]$ by a linear ramp of span s above baseline g_0 :

$$g = \frac{\text{gas}_{\text{raw}}}{B}, \quad G = \text{clamp}\left(\frac{g - g_0}{s}, 0, 1\right) \quad (1)$$

With $g_0 = 1.0$ and $s = 2.0$ the term saturates at three times baseline. The MQ-135 responds to a broad mixture of CO₂, NH₃ and assorted VOCs rather than to ethylene, so G is a commodity-agnostic *trend* indicator of headspace accumulation and is deliberately not converted into a ppm figure for any named gas — the way MQ-series arrays are used elsewhere, as correlated freshness proxies rather than calibrated gas meters [9], [12], [13], and consistent with volatile emission as a genuine spoilage pathway [6].

Only excursions *above* the optimal maximum contribute to the temperature term, since the interlock of Section V-C handles the cold side. Postharvest respiration rises geometrically with temperature, characterised by the Q_{10} coefficient [19], giving the relative rate increase τ and its normalised form $\hat{\tau}$:

$$\tau = Q_{10}(T)^{\max(0, T - T_{\max})/10} - 1, \quad \hat{\tau} = \frac{\tau}{1 + \tau} \quad (2)$$

The saturating map is parameter-free by design: a linear divisor would introduce a tuned constant with no physical justification and would clamp the most temperature-sensitive commodities

at 1.0 (Section VI-C). Under $\hat{\tau}$, steeper Q_{10} curves saturate sooner because the biology is more temperature-sensitive, not because of a fitted scale factor.

Humidity deviation is measured outside the band in either direction — both desiccation and condensation damage produce — and normalised by band width, giving R and the composite index:

$$R = \text{clamp}\left(\frac{\max(0, \text{RH}_{\min} - \text{RH}, \text{RH} - \text{RH}_{\max})}{\text{RH}_{\max} - \text{RH}_{\min}}, 0, 1\right) \quad (3)$$

$$\text{SRI} = \text{clamp}(w_1 \hat{\tau} + w_2 R + w_3 G, 0, 1)$$

with $w_1 + w_2 + w_3 = 1$; defaults $w_1 = 0.50$, $w_2 = 0.30$, $w_3 = 0.20$. These are engineering tunables, not biological constants, and live in configuration separately from the commodity profiles for exactly that reason. Keeping the composite additive and rule-based rather than learned follows the observation that compute-hungry inference suits resource-limited deployments poorly [14], [16], and that even hybrid learned systems fall back on interpretable rules for the actuation decision [17].

Q_{10} is itself not a physical constant: AH-66 [19] reports it per band, and variation within one commodity is substantial — onion ranges from 2.33 across 0–10°C to 1.14 across 10–20°C. The implementation stores the band table and selects the band containing T , returning the nearest band’s value outside the tabulated range — an acknowledged extrapolation, since AH-66 terminates at 20–25°C while kirana ambient reaches 30–40°C.

C. Actuation Decision Hierarchy

Three rules apply in strict priority order; the ordering is load-bearing and is enforced by test. Rule-based edge actuation has precedent in irrigation control [10] and multi-modal ambient-plus-gas sensing in environmental nodes [11], neither targeting produce-shelf preservation.

Rule 1 — Chilling-injury interlock (unconditional). An exhaust fan draws in ambient air; it cannot cool. If ambient is colder than the commodity tolerates, venting to reduce a high SRI would inflict precisely the injury the system exists to prevent — pitting in mature-green tomato below 13°C, irreversible reducing-sugar accumulation in potato below 3–4°C [19]. The fan is therefore forced OFF whenever $T \leq T_{\text{chill}}$, overriding everything else including a saturated gas reading. Where AH-66 records no chilling threshold — onion and leafy greens, neither chilling sensitive — the limit falls back to $T_{\min}$, guarding against freezing.

Rule 2 — Gas override. If $G \geq 0.90$ the fan is commanded ON and an alert opened regardless of composite SRI, because (3) is a weighted sum in which a saturated gas term contributes at most $w_3 = 0.20$ and cannot alone cross the 0.60 threshold. Yet gas at three times baseline while temperature and humidity sit inside their bands indicates localised decay in a subset of stock — the condition ambient averages conceal. The override restores sensitivity without inflating w_3 everywhere else.

Rule 3 — Hysteresis band. Otherwise, with previous state p , the fan turns on when $p = \text{off}$ and $\text{SRI} \geq \theta_{\text{on}} = 0.60$, turns off when $p = \text{on}$ and $\text{SRI} < \theta_{\text{off}} = 0.40$, and otherwise holds

p . The dead band prevents relay chatter near the threshold, which sensor noise alone suffices to produce (Section VI-D). Previous state is read from the persisted `fan_commanded` boolean, making the controller stateless and restart-safe.

An alert opens at $\text{SRI} \geq 0.70$, or unconditionally on gas override, if none is open; `peak_sri` rises with any higher reading and the episode resolves below 0.40, the asymmetry preventing spurious episodes from a reading oscillating about one boundary. Persistence and alerting are wrapped in a handler that logs and suppresses database exceptions — intentional, and covered by a test: an Atlas outage must degrade the system to “actuates but does not log”, never to “fails to actuate”.

D. Forecasting and Backend Surface

The dashboard projects SRI one hour ahead. Raw samples are smoothed into overlapping time-centred buckets — centres every 5 minutes over a 30-minute window, each averaging samples within ± 7.5 minutes — which tolerates the irregular arrival intervals Wi-Fi retries produce, unlike a fixed-count average. A least-squares slope m over the last four buckets is extrapolated from the final bucket mean $\bar{y}_K$ as $\text{SRI}(\Delta) = \max(0, \bar{y}_K + m\Delta)$. No ceiling is imposed: a projection exceeding 1.0 usefully signals the threshold will be crossed inside the horizon. A first-order fit is deliberate: simple models remain competitive for short-horizon spoilage trend prediction, with temperature the dominant driver [18]. A dual guard suppresses the forecast unless at least four distinct raw readings *and* four non-empty buckets exist, since four samples clustered in one bucket would satisfy a naive count while carrying no trend information.

The service exposes fourteen endpoints across ingest, device and assignment management, calibration, commodity lookup, history, alerts, forecast and health, with a plain HTML/JavaScript dashboard polling every 45 seconds. All timestamps pass through a UTC-coercing Pydantic type, since the Motor client is not constructed timezone-aware and a plain `datetime` annotation serialises without an offset — a defect that appeared as a 5.5-hour displacement between the historical and forecast traces before the coercion was applied.

VI. RESULTS AND EVALUATION

A. Scope and Methodology

What was *not* measured matters as much as what was. No physical shelf has been deployed and no readings from assembled hardware logged, so this paper reports **no** sensor accuracy, **no** pilot-store spoilage-reduction figure and **no** measured power draw.

What is reported falls into three reader-verifiable categories. (i) *Automated verification*: an executable test suite over the decision logic. (ii) *Analytical characterisation*: SRI values computed by the production code from the seeded profiles of Table I, deterministic consequences of (3) and the cited constants that reproduce on any host. (iii) *Simulation-driven evaluation*: the controller and forecaster exercised over synthesised ambient trajectories, which are inputs and are labelled

TABLE II
SRI TERM DECOMPOSITION, TOMATO (BASELINE $B = 100$)

T	RH	gas	ΔT	Q_{10}	τ	$\hat{\tau}$	R/G	SRI
20.0	92	100	0.0	2.38	0.000	0.000	0.00/0.00	0.000
26.0	92	100	5.0	1.95	0.396	0.284	0.00/0.00	0.142
20.0	92	280	0.0	2.38	0.000	0.000	0.00/0.90	0.180
34.0	60	150	13.0	1.95	1.383	0.580	1.00/0.25	0.640
31.0	70	300	10.0	1.95	0.950	0.487	1.00/1.00	0.744

TABLE III
SRI VS. AMBIENT TEMPERATURE (RH = 90%, $G = 0$)

Commodity	20°C	25°C	30°C	35°C	40°C
Tomato	0.000	0.117	0.226	0.304	0.359
Onion	0.415	0.440	0.463	0.484	0.504
Potato	0.127	0.178	0.222	0.259	0.292
Leafy greens	0.686	0.721	0.745	0.762	0.774

synthetic wherever used — every SRI value, actuation, alert transition and forecast reported from them comes from the production code path, not a model of it. Synthetic-input evaluation is established practice here (both [17] and [18] validate on generated data) but substantiates control-logic correctness, not field efficacy, and is not claimed to. Measurements were taken on an x86-64 Linux host under CPython 3.11.15 with MongoDB replaced by an in-process mock.

B. Verification and Analytical Checks

Thirty-one tests execute against the decision logic and all pass: 15 over the spoilage service (SRI derivations, resolution chain, interlock precedence, hysteresis, alerts, write resilience), 6 over forecasting, 5 over the readings API, 4 over device and assignment management, and 1 over seed idempotency. Expected SRI values are hand-derived in comments beside each assertion rather than captured from a previous run, so a formula regression fails the test rather than silently redefining it.

Table II decomposes (3) for tomato across representative kirana conditions, showing every intermediate quantity; each row is reproducible by hand from Table I. The third row isolates the case motivating the gas override: gas at $2.8\times$ baseline with temperature and humidity both inside their optimal bands yields $\text{SRI} = 0.180$, far below θ_{on} . Without Rule 2 this condition would not actuate the fan at all.

C. Cross-Commodity Sensitivity

Table III evaluates all four commodities at fixed RH = 90% and clean-air gas. Three findings follow, of which the second and third are as much diagnoses of the formulation’s limits as they are results.

First, the ordering matches postharvest expectation. Leafy greens carry the highest risk at every temperature, consistent with a respiration rate of 229.5 mg CO₂/kg/h at 20°C against tomato’s 34.5 [19], and at 20°C — ordinary kirana ambient — they already exceed $\theta_{\text{on}} = 0.60$, so the system commands continuous ventilation. That is the correct conclusion: leafy greens cannot be held at ambient without rapid quality loss.

Second, the saturating normalisation earns its place: leafy greens reach $\tau = 3.37$ at 20°C and $\tau = 12.20$ at 35°C , which a linear divisor would clamp alike to 1.0, rendering the entire operating range indistinguishable, whereas under $\hat{\tau}$ the values stay separable and monotone.

Third — a limitation surfaced by the characterisation rather than a positive result — onion’s SRI is almost entirely humidity-driven. Its optimal RH band is 65–75%, so at RH = 90% the deviation is 15 points against a 10-point band, saturating R and contributing a fixed 0.30 at every temperature; the remaining variation across a 20°C span is only 0.089, because onion’s Q_{10} of 1.14 is nearly flat. The composite is a poor discriminator for onion — a property of the commodity rather than a defect of the code, since onion is genuinely shelf-stable in temperature and vulnerable to humidity [19]. Monitoring onion alone would need re-weighted w_i , which the weights being configuration rather than code makes possible. A related caution applies to both: AH-66 gives each an optimal maximum of 0°C , so ΔT at kirana ambient is $20\text{--}40^\circ\text{C}$, far outside the measured range, and their absolute values are ordinal rankings, not calibrated probabilities.

D. Control Behaviour

Sweeping temperature against the production decision function with SRI and gas both at maximum — the strongest competing demand for ventilation — the interlock activates at exactly the declared limit for every commodity: 13.0°C for tomato and 2.0°C for potato from explicit thresholds, 0.0°C for onion and leafy greens by $T_{\min}$ fallback. Precedence is confirmed: at $T = 12^\circ\text{C}$ with a saturated gas term the command is OFF, so Rule 1 dominates Rule 2 — a test pins this, since reordering would vent cold air into mature-green tomato. For onion and leafy greens the 0°C limit will essentially never be reached in an Indian kirana store, so the interlock is exercised in deployment only for tomato and potato.

The hysteresis dead band was evaluated by holding a simulated 6-hour trajectory near θ_{on} and injecting zero-mean Gaussian sensor noise ($\sigma = 0.5^\circ\text{C}$, 2% RH, 5 ADC counts), sampling every 2 minutes over 10 seeds of 181 samples each. Realised SRI spanned 0.579–0.655, straddling the 0.60 threshold. A single-threshold controller produced a mean of **20.4** relay transitions per run (range 15–26); the hysteresis controller produced **1.0** in every run — a **95.1%** reduction, or roughly 30,000 noise-induced relay operations per year against about 1,500. For a low-cost mechanical relay that is the difference between a component that survives deployment and one that does not, achieved by two configuration constants rather than by filtering.

E. Forecast Accuracy and Computational Cost

The forecaster was evaluated over a simulated 12-hour shop day for tomato — temperature rising from 24°C to a 33°C peak, RH falling from 88% to 66%, gas accumulating with an accelerating second-half slope — sampled every 2 minutes, with forecasts at 20-minute intervals compared against the SRI

subsequently realised. Across 31 invocations and 372 predicted points, mean absolute error over the 60-minute horizon was **0.0085** SRI units, falling to **0.0050** over the first 30 minutes (186 points). Worst-case error was 0.136 at the horizon edge, where linear extrapolation overshoots the afternoon inflection — the expected failure mode of a first-order fit against a curving trajectory. The cold-start guard was confirmed: with three readings the service returns `insufficient_data` rather than extrapolating from a two-point slope. These figures are not field accuracy: real kirana conditions contain door-opening transients, restocking and stock turnover a 30-minute linear fit cannot anticipate.

The decision core — SRI, gas term and the three-rule hierarchy — executes in **3.69 μs** median (4.33 μs p95, 9.39 μs p99, $n = 2,000$). The full ingest path including document construction, persistence and alert evaluation completes in **2.80 ms** median (5.20 ms p95, $n = 361$), excluding network and Atlas round-trip and therefore a lower bound on deployed latency. The decision-core figure is the load-bearing one: at under 4 μs with no matrix arithmetic, no learned parameters and a working set of roughly a dozen scalars, the computation is small enough to port to the ESP32 without approximation — the quantitative basis for the migration argued in Section IV-B. By comparison, the TinyML survey of [16] reports on-device inference trading roughly ten percentage points of accuracy for latency reduction; this design avoids that trade by not requiring inference. Over the same shop day the controller held the fan on for 64.5% of samples with a single relay transition at minute 256 when SRI crossed 0.600, and one alert opened, peaking at SRI = 0.715, correctly linked to its triggering sample.

F. Bill of Materials and Cost Positioning

The node was procured in Bangalore in 2026 for INR 1,155: ESP32 38-pin USB-C board (340), DHT22 AM2302 sensor (120), MQ-135 module (110), optoisolated 5 V relay (80), 80 mm 5 V fan (80), jumper wires (50), stranded hookup wire (100), GL-12 terminal strip (55), perfboard $\times 2$ (80) and 2-pin connectors $\times 2$ (140), all in INR — within the sub-INR 2,000 target of Section II; soldering equipment (INR 160) is excluded as a one-time workshop tool. Against the INR 60,000–70,000 ESP32/DHT22/MQ-135 controlled-environment onion storage unit of [7], the closest comparable low-cost system in the literature, this is a reduction of roughly **52–61 $\times$** — a comparison of cost position, not capability, since [7] adds UV-C disinfection and active environmental control this design does not attempt. The INR 9.55 per month energy cost stated in the abstract is a design-stage estimate from nameplate ratings, near 1.9 W at a tariff of about INR 7 per kWh; it has **not** been measured, because the node is not instrumented.

G. Limitations

In descending order of how much they constrain the claims above. (i) *No physical deployment* — the dominant limitation: sensor accuracy, calibration drift, MQ-135 cross-sensitivity and warm-up under humidity, fan effectiveness at displacing

headspace gas, and spoilage reduction are all unmeasured; nothing above substitutes for a deployed shelf. (ii) *Decision computed server-side*, so the offline-autonomy property of gap II-B is argued from the decision function's size and dependency set rather than demonstrated. (iii) *Q_{10} extrapolation* beyond the 20–25°C ceiling of the tabulated data [19], so values at 30–40°C should be read ordinally. (iv) *Unvalidated weights* — w_1, w_2, w_3 and both thresholds are design choices without empirical grounding, suiting tomato and potato better than onion. (v) *RH-band proxies* from ripening and curing conditions for tomato and potato, with spinach as the leafy-green proxy. (vi) *Non-specific gas term* — the MQ-135 responds to a broad mixture rather than to ethylene, and one uncalibrated baseline per device is assumed stable over time. (vii) *Simulated inputs* for the hysteresis, forecast and duty-cycle figures.

VII. CONCLUSION

This paper presented a smart shelf microclimate monitoring and spoilage risk analytics system targeting a segment the postharvest IoT literature has largely bypassed: the Indian MSME kirana retailer, for whom RFID multisensor platforms [2], [4], [5] and controlled-environment storage [7] are economically inaccessible, and for whom passive monitoring nodes offer no protective action [8], [9].

Four contributions are offered: a composite Spoilage Risk Index fusing Q_{10} -scaled temperature excess, humidity band deviation and normalised gas accumulation into one interpretable score, with a parameter-free saturating normalisation preserving discrimination across commodities differing by an order of magnitude in temperature sensitivity; an actuation hierarchy placing an unconditional chilling-injury interlock above a gas override above a hysteresis band, a precedence necessary rather than incidental since an exhaust fan draws ambient air and can inflict the injury it is deployed to prevent; an architecture treating postharvest biology [19] as versioned, citation-bearing reference data rather than program constants; and a bill of materials of INR 1,155, some 52–61× below the nearest comparable system [7].

The evaluation substantiates control-logic correctness — 31 passing tests, interlock activation verified at the declared limit for all four commodities, a 95.1% reduction in noise-induced relay transitions, and a decision core under 4 μ s — but not field efficacy, and the paper does not claim otherwise. No hardware has been deployed.

Three lines of work follow. The immediate priority is physical deployment: assembling the node, characterising MQ-135 drift and humidity cross-sensitivity in situ, and measuring power draw against the INR 9.55 per month estimate. The second is porting the decision function to ESP32 firmware with NVS-cached profile fields, closing the offline-autonomy gap by demonstration rather than argument; the measured cost of the decision core makes this a porting exercise, not a redesign. The third is empirical tuning of the weights and thresholds against observed spoilage outcomes — and because every reading is persisted with its full resolution context, that history is also the training corpus a learned successor would

require [14], making the rule-based design a deliberate first stage rather than a permanent ceiling.

REFERENCES

- [1] V. M. Indumathi, M. Malarkodi, and B. Navaneetham, "Factors influencing shrinkage of vegetables in select retail outlets in Coimbatore city, Tamil Nadu," *Int. J. Farm Sci.*, vol. 10, pp. 88–92, 2020.
- [2] C. Song, Z. Wu, J. Gray, and Z. Meng, "An RFID-powered multisensing fusion industrial IoT system for food quality assessment and sensing," *IEEE Trans. Ind. Informat.*, 2024, doi: 10.1109/TII.2023.3262197.
- [3] E. Claucherty, D. Cummins, A. Rossi, and B. Aliakbarian, "Effect of beverage composition on radio frequency identification (RFID) performance using polyethylene terephthalate (PET) bottles for smart food packaging applications," *Foods*, vol. 13, no. 5, p. 643, 2024, doi: 10.3390/foods13050643.
- [4] Z. Cao, Z. Wu, and J. Gray, "RFID tag-integrated multi-sensors with AIoT cloud platform for food quality analysis," *Electronics*, vol. 15, no. 1, p. 106, 2026, doi: 10.3390/electronics15010106.
- [5] C. Song and Z. Wu, "Artificial intelligence-assisted RFID tag-integrated multi-sensor for quality assessment and sensing," *Sensors*, vol. 24, no. 6, p. 1813, 2024, doi: 10.3390/s24061813.
- [6] R. S. Andre, R. Schneider, G. R. DeLima, L. Fugikawa-Santos, and D. S. Correa, "Wireless sensor for meat freshness assessment based on radio frequency communication," *ACS Sensors*, vol. 9, no. 2, pp. 631–637, 2024, doi: 10.1021/acssensors.3c01657.
- [7] S. Kumar et al., "An IoT-based controlled environment storage for prevention of spoilage of onion (*Allium cepa*) during post-harvest with UV-C disinfection," arXiv:2601.10745, 2026.
- [8] V. Sámano-Ortega, O. Arzate-Rivas, J. Martínez-Nolasco, J. Aguilera-Álvarez, C. Martínez-Nolasco, and M. Santoyo-Mora, "Multipurpose modular wireless sensor for remote monitoring and IoT applications," *Sensors*, vol. 24, no. 4, p. 1277, 2024, doi: 10.3390/s24041277.
- [9] A. Bhattacharjee, N. A. Biswas, K. A. Shahriar, and K. B. Salim, "IoT-based cost-effective fruit quality monitoring system using electronic nose," arXiv:2512.08753, 2025.
- [10] M. S. Munir, I. S. Bajwa, A. Ashraf, W. Anwar, and R. Rashid, "Intelligent and smart irrigation system using edge computing and IoT," *Complexity*, vol. 2021, art. 6691571, 2021, doi: 10.1155/2021/6691571.
- [11] P. Wiese, V. Kartsch, M. Guernandi, and L. Benini, "A multi-modal IoT node for energy-efficient environmental monitoring with edge AI processing," in *Proc. IEEE Conf.*, 2025.
- [12] M. Trebar, A. Žalik, and R. Vidrih, "Assessment of 'Golden Delicious' apples using an electronic nose and machine learning to determine ripening stages," *Foods*, vol. 13, no. 16, p. 2530, 2024, doi: 10.3390/foods13162530.
- [13] K. S. Rahman, M. M. Salehin, R. Roy, J. Bhadra Swarna, M. R. Islam Rakib, C. K. Saha, and A. Rahman, "Prediction of mango quality during ripening stage using MQ-based electronic nose and multiple linear regression," *Smart Agric. Technol.*, vol. 9, p. 100558, 2024, doi: 10.1016/j.atech.2024.100558.
- [14] M. Rashvand, Y. Ren, D.-W. Sun, J. Senge, C. Krupitzer, T. Fadji, M. S. Miró, A. Shenfield, N. J. Watson, and H. Zhang, "Artificial intelligence for prediction of shelf-life of various food products: recent advances and ongoing challenges," *Trends Food Sci. Technol.*, vol. 159, p. 104989, 2025, doi: 10.1016/j.tifs.2025.104989.
- [15] A. U. Haque, M. A. Al Haque, A. Alabdulatheem, A. Al Mulla, N. Almulhim, and R. Srinivasagan, "Machine learning-based shelf life estimator for dates using a multichannel gas sensor: enhancing food security," *Sensors*, vol. 25, no. 13, p. 4063, 2025, doi: 10.3390/s25134063.
- [16] S. Heydari and Q. H. Mahmoud, "Tiny machine learning and on-device inference: a survey of applications, challenges, and future directions," *Sensors*, vol. 25, no. 10, p. 3191, 2025, doi: 10.3390/s25103191.
- [17] I. Singh, D. Chawla, A. Garg, S. Mangal, P. Gupta, K. Agarwal, N. S. Khalsa, and N. Patel, "Interpretable hybrid deep Q-learning framework for IoT-based food spoilage prediction with synthetic data generation and hardware validation," arXiv:2512.19361, 2025.
- [18] R. B. M. and R. S., "Sensor data simulation and predictive modeling for food spoilage risk in green IoT-enabled supply chains," *Int. J. Adv. Signal Image Sci. (IJASIS)*, pp. 1593–1603, 2026, doi: 10.29284/ijis9z406.
- [19] K. C. Gross, C. Y. Wang, and M. Saltveit, Eds., *The Commercial Storage of Fruits, Vegetables, and Florist and Nursery Stocks*, Agriculture Handbook 66, USDA Agricultural Research Service, 2016, doi: 10.22004/ag.econ.348552.